PROGRAMMAZIONE 09.03

COSTRUTTORI

Costruttore: metodo speciale che ha lo scopo di rendere utilizzabile un oggetto quando è creato

- hanno il nome della classe
- non hanno valore di ritorno
  Il compilatore crea sempre un costruttore di default

es di inizializzazione:
`class Stack {`
`public`
`Stack(int s ): TOS(0), size(s), buffer(new data[s]) -> i due punti inizializzano una lista di oggetti`
`protected:`
`int Tos;`
`int size;`
`int buffer; -> è buona pratica inizializzare gli oggetti nello stesso ordine della lista`
`}`
Inizializzazione costruttore == definizione + inizializzazione
es:
`class X {`
`public:`
`X( ) { } // i=4, j=5;`
`X(int ni) : i(ni) {} //j=5`
`int i=4;`
`int j{5} //alternative syntax`
`}`
creo un costruttore e dopo dichiaro le variabili all'interno del costruttore

Buona pratica:
bisogna dichiarare certi costruttori di tipo esplicito, aggiungendo `explicit` cosi che il compilatore faccia delle conversioni di tipo solo quanto è richiesto dal programmatore (quando c'è `explicit`). L' `explicit` impedisce al compilatore di fare conversioni di tipo implicitamente. Se il costruttore ha più argomenti mettere `explicit` **non serve a nulla**

esempio costruttori espliciti:
`class A {`
`public:`

```
A();
};

class B {
public:
explicit B(int x=0, bool b=true);
};

class C {
public:
explicit C(B ab=B());
};

void doSomething(B objB);
void doSomethingC(C objC);

B objB1; // x and b args set to default

doSomething( objB1 ); // OK

B objB2( 28 ); // OK, b arg is set to default

doSomething( objB2 ); // OK
doSomething(28); // BAD: we need a B obj, and we do not allow implicit conversion


doSomething(B(28)); // OK

C objC1;
doSomethingC(objC1);
doSomethingC(objB2); // BAD, we need a C object, and we do not allow implicit
conversion

doSomething("foo"); // BAD, thanks the compiler for not allowing it
```

quindi:
è preferibile usare costruttori espliciti, cosicché i tipi siano convertiti solo quando vuole il programmatore (non ci si fida del compilatore)

es: riaprire il progetto dell'ultima volta e vedere dove bisogna aggiungere `explicit`

DELEGATE CONSTRUCTORS
(introdotto in C++ 11)
un costruttore può chiamare un altro costruttore.
Scriviamo il codice comune del costruttore in uno solo e chiamandolo negli altri costruttori non dovremmo riscriverlo.

es:

```
class Sales_data {
public:
// non delegating constructor, uses init. list
Sales_data(std::string s, unsigned cnt, double price):
bookNo(s), units_sold(cnt), revenue(cnt*price) { }    // remaining constructors
all delegate to another constructor

Sales_data(): Sales_data("", 0, 0) {}
Sales_data(std::string s): Sales_data(s, 0,0) {}
Sales_data(std::istream &is):
Sales_data() {}
// other members
// ...
};
```

DISTRUTTORI

Oltre ad avere un metodo di inizializzazione di oggeti, ha un metodo che "fà la pulizia" quando un oggetto non serve più
Caratteristiche:

- chiamato con tilde + nome classe (es: `~Stack();`)
- non ha valore di ritorno
- non può essere sovraccaricato
- non può prendere argomenti
- è chiamato automaticamente quando un oggetto è distrutto
- massimo 1 per classe

Quando è necessario un distruttore?
Se la classe ha acquisito risorse quando l'oggetto è stato creato che devono essere liberate quando esso è distrutto
(es: memoria legata all'oggetto, le connessioni al database, i mutex...)

es:

```
class IntStack {
public:
IntStack(int s);
~IntStack(); //..
protected:
int TOS;
```

```
int size;
int* buffer;
};
```

```
// C’tor allocates memory
IntStack::IntStack(int s) {
TOS=0;
size=s;
buffer = new int[size];
}
// D’tor has to  release memory
```

```
IntStack::~IntStack() {
delete[] buffer;
}
```

ALTRI ASPETTI E MEMBRI DEL DESIGN DELLE CLASSI

- **THIS**
  keyword usata quando c'è riferimento a metodi o attributi

A meno che serve, NON SCRIVERE this
E quando serve?
per es. quando metodi e oggetti hanno lo stesso nome per distinguerli

Il "vero scopo" del this è passare gli indirizzi di oggetti agli altri oggetti->Questo serve per implementare lo schema di **inversione di responsabilità**

- **MEMBRI STATICI**
  può essere che ci sia uno stato che sia condiviso da valori dello stesso tipo, o che certi metodi siano divisi tra tutte le istanze
  keyword: `static`
  - estende il tempo di vita di un attributo, cosi che sia condiviso tra i valori
  - per i metodi significa che esso può operare sulle stesse istanze della classe

il il membro di una funzione `static` può essere invocato senza riferimento a un oggetto
es per attributi:

```
file .h:
class Point {
public:
Point() {
x=y=0;
n++;
}
```

```
~Point() {
n--;
}
int count() const {
return n;
}
// ...
private:
int x,y;
static int n; // declaration
};
```

```
file .cpp
// definition: must be in namespace scope
int Point::n = 0;
int main() {
Point a,b;
cout << “n: “ << a.count() << endl;
cout << “n: “ << b.count() << endl;
}
```

es. per metodi:

```
file.h
class Point {
public:
Point() {
x=y=0;
n++;
}  ~Point() {
n--;  }
static int n; // declaration
static float distance(
const Point& a,   const Point& b)
{
//...calc distance
}
// ...
private:  int x,y;
};
```

```
file .cpp
// definition: must be in namespace scope
```

```
int Point::n = 0;    int main() {
// access static members even before the creation of any instance of  the class
cout << “n: “ << Point::n << endl;
Point a,b;  // set a and b coordinates
Point::distance(a,b);
}
```

Gli attributi statici, quando sono inizializzati, vengono dichiarati senza che il programmatore lo debba fare

- **FRIEND**
  la maggiore parte del lavoro del programmatore è decidere dell'interfaccia delle classi (cosa è pubblico e cosa è privato), ma questo può diventare scomodo
  Noi possiamo decidere che certe funzioni o oggetti sono "amiche" della classe, cosicché ci possano accedere, rompendo il lavoro di incapsulamento, senza che le funzioni per es. debbano prima usare una funzione getter

keyword: `friend`

caso d'uso di friend: operazioni di input/ouput, salvando certi output per es.

es:
versione 1:

```
class Point{
private:
int x,y;
public:
friend bool operator == (Point a,   Point b);
Point() : x(0), y(0) {};
//...
};
bool operator == (Point a, Point b) {
if ( ( a.x != b.x ) ||   ( a.y != b.y ) )
return false;
else
return true;
}

int main() {
Point p, q;
//...
if (p == q)
std::cout << "p and q are equal"  << endl;
```

```
return 0;
}
```

versione 2

```
class Point{
private:
int x,y;
public:
bool operator == (Point right);
Point() : x(0), y(0) {};
//...
};

bool Point::operator == (Point right) {
if ( ( this->x != right.x ) ||   ( this->y != right.y ) )
return false;
else
return true;
}

int main() {
Point p, q;
//...
if (p == q)
std::cout << "p and q are equal" << endl;
return 0;
}
```

CLASSI INTERNE

Classi dichiarate all'interno di altre classi

Si vuole poter istanziare gli oggetti di una classe solamente all'interno di una certa classe.
Le classi interne mi permettono di programmare in O-O anche all'interno delle classi, semplificando così il programma.
Le classi contenute sono considerate come classi "amiche" della classe contenente, cosicché le prime possano ignorare i livelli di accesso della seconda

es:

```
int x,y; // globals
class enclose { // enclosing class
private:
int x; // NOTE: private members
```

```
static int s;
public:
struct inner { // nested class
void f(int i) {
x = i; // Error: can't write to non-static  enclose::x without instance
s = i; // OK: can assign to the static enclose::s  :
:x = i; // OK: can assign to global x
y = i; // OK: can assign to global y
}
void g(enclose* p, int i) {
p->x = i; // OK: assign to enclose::x
}
};
};
```

es:

```
class Algorithm{
public:
class AlgorithmResults{
friend class Algorithm;
public:
void readAlgorithmResult();
private:
void writeAlgorithmResult();
};
void calculate(AlgorithmResults& results, Arguments...){
//calculate stuff  results.writeAlgorithmResult(results);
}
};
```

es:

```
class List {
public:
List(): head(nullptr), tail(nullptr) {}
private:
class Node {
// not exposed to users of List class  public:  int data;  Node* next;  Node*
prev;
};
Node* head;
Node* tail;
};
```

PRINCIPIO DI SINGOLA RESPONSABILITA

Ogni oggetto all'interno di un sistema deve avere una singola responsabilità, quindi una classe deve avere un unico motivo per cambiare
Questo rende più chiara l'evoluzione del software nel tempo
Se una ha classe ha tante responsabilità, ci sono tante dipendenze tra le altre classi, e ciò non rende eseguibile il principio di aperto chiuso[1]

Come facciamo a sapere se stiamo seguendo questo principio?

- quanti motivi ho per cambiare la classe? -> devo averne solo uno
- non devo accoppiare due oggetti che cambiano per motivi diversi in tempi diversi
- la seguente linea deve aver senso (trucco linguistico in inglese): "The (class name) (write method here) itself."

CONST CORRECTNESS

Meccanismo di limitazione semantica t.c. un oggetto non cambia nel tempo, richiedendo al compilatore di controllare ciò
keyword: `const`, si usa per:

- per variabili globali al di fuori della funzione cosicché NON varino durante l'esecuzione di essa
- per fissare attributi statici o meno
  es:
  `class VideoFrame {`
  `private:  static const int PALFrameRate;`
  `...`
  `};`
- per fissare puntatori e riferimenti
  es:
  `char greeting[] = “Hello”;`
  `char* p = greeting; // nothing is constant`
  `const char* p = greeting; //non-const pointer, const data`
  `char* const p = greeting; // const pointer, non-const data`
  `const char* const p = greeting; // everything is const`

  con i **riferimenti** la costanza può essere soltanto **a sinistra della &**
- per dichiarare costanti dei metodi per limitare errori
  es:
  `class Rational {`
  `//...`

```
};
const Rational operator*(  const Rational& lhs,  const Rational& rhs  );
Rational a,b,c;  // let’s say we missed an =  to make a comparison...
(a * b)=c; // it’s now illegal thanks to const return value !
```

- quando il return value è un riferimento
  es:
  ```
  class Person {
  public:
  string& badGetName() {
  return name;
  }
  //...
  private:
  string name;
  };
  void myCode(Person& p) {
  p.badGetName() = “Igor”; // can change the name  attribute of Person
  }
  ```
- per dichiarare un metodo stesso costante, e quindi tutti i suoi attributi
  questo è un passo essenziale per sovraccaricare un metodo
  es:

```
class TextBlock {
  public:
      const char& operator[](size_t pos) const {
          return text[pos];
      }
      char& operator[](size_t pos) { // has to be reference
          return text[pos]; // to be modifiable:
        } // C++ returns by value !
    private:  string text;
};
```

il compilatore vede se è possibile toccare gli attributi del metodo, se non è il caso capisce che **il metodo è costante**
si può anche chiedere al compilatore di non considerare alcuni attributi nel controllo della costanza, per esempio se ho bisogno di modificarli
keyword: `mutable`
es:
```
class TextBlock {
public:
size_t length() const;
```

```
private:
string text;
mutable size_t length;
mutable bool isValidLength;
};
size_t TextBlock::length() const {
if(!isValidLength) {
length=text.size();
isValidLength=true;
}
return length;
}
```

---

1. programma aperto a estensioni ma chiuso a modifiche al programma preesistente ↩